

Ray tracing on CPU

Bc. Josef Jech
FIT CTU
Prague, Czechia
josefjech.mail@gmail.com

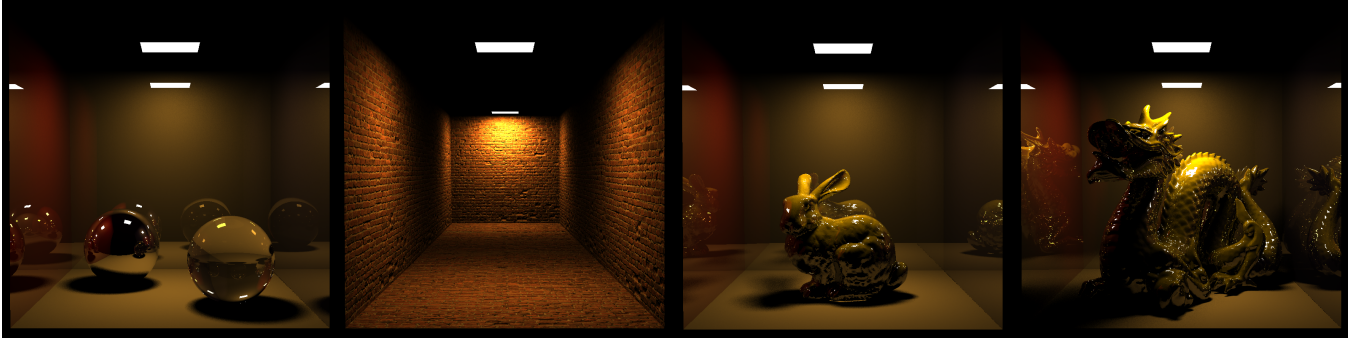


Figure 1: Ray tracer showcase of renders using Phong illumination

CCS Concepts

• **Computing methodologies** → **Ray tracing; Reflectance modeling; Visibility; Texturing.**

1 Introduction

As a semester project for the Computer Graphics 1 course, I prepared a ray tracer in C++ that runs on a single CPU thread. Ray tracing is a method of approximating the global illumination of a scene, where for each pixel we shoot a ray and search for the closest triangle in the scene that the ray intersects. Because of this, there is no need to deal with visibility algorithms.

From the intersection point with the triangle, a ray is then sent to all lights to check if something is blocking its path. If not, it calculates the light contributions to the pixel color. It calculates its BRDF and recursively shoots additional rays that simulate reflections and transparency, and their color is added to the pixel color. For such a global illumination method, there are many improvements, and I will discuss some implemented here.

2 Implementation

The ray tracer shoots primary, shadow, reflected, and refracted rays. It can basically work with area lights, meshes in the Obj format, and materials in the Mtl format. The color is evaluated by the Phong illumination model. Using the stored UV coordinates, it can map textures, which it filters using bilinear interpolation. It saves detailed statistics to output/times.log and is configurable using config.json.

2.1 Improvements

GGX

An illumination model based on microfacet BRDF that improves the previous Cook-Torrance model and is used for PBR. It is based

on three terms: F, G, and D. These model F - the Fresnel effect, G - the shadowing and masking of microfacets, and D - the statistical normal distribution function of microfacets.

$$f_r(\omega_i, \omega_o) = \frac{D(h) F(\omega_i, h) G(\omega_i, \omega_o)}{4 (\mathbf{n} \cdot \omega_i) (\mathbf{n} \cdot \omega_o)} \quad (1)$$

For the F term, it uses Schlick's approximation, which is a faster calculation of the Fresnel equation. For the G term in the Smith variant, it uses the Schlick-GGX G1 function:

$$G_{\text{Smith}}(\omega_i, \omega_o) = G_1(\omega_i) G_1(\omega_o) \quad (2)$$

$$G_1(\omega) = \frac{\mathbf{n} \cdot \omega}{(\mathbf{n} \cdot \omega)(1 - k) + k} \quad (3)$$

And the D term using the GGX distribution:

$$D_{\text{GGX}}(h) = \frac{\alpha^2}{\pi [(\mathbf{n} \cdot h)^2 (\alpha^2 - 1) + 1]^2} \quad (4)$$

The parameter α represents the squared roughness of the surface. We calculate this value from the Phong shininess exponent h as follows:

$$\text{roughness} = \sqrt{\frac{2}{h + 2}} \quad (5)$$

$$\alpha = \text{roughness}^2 \quad (6)$$

To switch the BRDF from Phong to GGX, it is sufficient to set GGX:true in the configuration file.

Fresnel equation & TIR

The Fresnel term is used in the renderer as a coefficient determining what part of the incident light reflects from the surface depending on the view angle. For transparent materials, this coefficient controls the energy distribution between the reflected and refracted ray. For opaque materials, it then defines the intensity of the specular

reflection. For an efficient calculation of this phenomenon, Schlick's approximation is implemented:

$$F(\omega_i, h) = F_0 + (1 - F_0) (1 - (\omega_i \cdot h))^5 \quad (7)$$

However, if a situation occurs for the reflected ray where the refracted ray does not reach the surface and gets trapped, Fresnel ensures that all light reflects from the surface.

Mipmapping

Mipmapping is the creation of a texture cascade (levels), where each subsequent level has half the side length, dividing until a level with 1 texel is created. It prevents texture aliasing. For mipmap filtering, it is necessary to interpolate texels from the bilinearly interpolated previous and next texture levels that we calculated.

Instead of a method for finding the level based on the derivative of neighboring pixels, I used the "footprint estimation" method. This method describes how many texels at a given distance would fit into one screen pixel with a level 0 texture. To get the level, *log ratio* is calculated. For this, all these parameters are needed:

- texScale - how dense the texture is on the triangle area
- pixelSpreadAngle - the view angle of a single pixel
- footprintWorld - the size of the triangle per one pixel (objects in the distance are larger relative to a pixel)
- ratio - how many texels from the level 0 texture correspond to one pixel on the monitor

Normal mapping

The normal calculation is done by a transformation from the normal map into tangent space, which is defined by the original normal, tangent, and bitangent. At each point, a normal and its tangent obtained from Gram-Schmidt orthogonalization are stored.

Then it is necessary to create a TBN matrix for a specific point. The normal N and tangent T are interpolated for smooth shading between the triangle vertices. The problem is that during interpolation, the orthonormality between N and T is lost, and it is necessary to orthogonalize them again. Based on N and T, the bitangent B is created, and then we just need to multiply the TBN matrix by the normal from the map.

Gamma correction & Tone Mapping

Because changes in intensity are not perceived linearly, but colors are calculated that way, it is necessary to first perform linear operations with colors and raise the resulting pixel to the power of 1/2.2. Textures mostly already have gamma correction applied, so they need to be raised to the power of 2.2 first.

Even before gamma correction, it is possible to apply Reinhard Tone Mapping, which better distributes colors from HDR into the resulting 0-1 range. Applied to GGX, if we used the gammaClamp function instead of postProcessPixel.

The second option is an approximation of the ACES color space using a filmic curve, which is implemented in GGX as postProcessPixel, and we can change the exposure for it. Ref: <https://knarkowicz.wordpress.com/2016/01/06/aces-filmic-tone-mapping-curve/>.

3 Kd-tree

To speed up finding the intersection with a triangle, acceleration structures are used, and I implemented a Kd-tree. It is a spatial partitioning structure that divides space in nodes by alternating the splitting plane along the x, y, z axes. I will describe the naive approach to tree building using median space division and then using SAH.

3.1 Median Build

The basic (naive) tree building runs recursively using the median split method. The splitting axis (X, Y, or Z) regularly alternates in each node depending on the current depth in the tree (typically as *depth mod 3*).

To determine the exact position of the splitting plane, the centroids (centers) of individual triangles are then used. The triangles are sorted by their centroid coordinates on the currently selected axis, and the middle index (median) is chosen, which divides the geometry into two halves for the left and right subtrees. The recursive division stops and a leaf node is created in three cases: if the maximum tree depth is reached, if the number of triangles in the node drops below the allowed minimum, or if it is detected that the triangles have clustered and no triangles would be removed by dividing.

3.2 SAH build

The improved method using the Surface Area Heuristic (SAH) tries to minimize the computational cost of future tree traversal. For each node, it analyzes the splitting along all three axes. The boundaries (minimum and maximum) of the AABB of individual triangles are used in the implementation as candidates for the splitting plane.

For each candidate, it counts how many triangles would fall into the left and right child, and the splitting cost (C) is estimated according to the formula:

$$C = C_{trav} + \frac{S_L}{S_{node}} N_L C_{isect} + \frac{S_R}{S_{node}} N_R C_{isect} \quad (8)$$

where C_{trav} is the constant cost of traversing an internal node, and C_{isect} is the cost of calculating the intersection with a triangle. The fraction represents the surface area ratio of the left/right node ($S_{L/R}$) to the surface area of the whole current node (S_{node}), and $N_{L/R}$ is the number of triangles in the given branch. Then, the splitting plane with the lowest cost is selected. However, if this minimum cost is still higher than creating a leaf itself (the cost corresponding only to intersections: $N \cdot C_{isect}$), the division does not happen, and the node becomes a leaf.

3.3 Traversal

Tree traversal is implemented iteratively using a pre-allocated stack. In the first step, the ray intersection with the main AABB of the whole tree is tested, which gives the distances of the ray entering the tree (t_{min}) and exiting (t_{max}).

During the traversal, the intersection distance of the ray with the splitting plane (t_{split}) is calculated for each internal node. Based on the ray direction, the algorithm evaluates which child is closer and which is further, and handles three situations:

Table 1: Ray tracer performance results

Scene/Measures (times in ms)	Render time	kRays/s	Build time	#Nodes	Avg tree depth	Avg tris/leaf	mem [KB]	Avg Incid. trav.	Avg ray trav. time	Avg Trav. Depth	Time vs BASE
<i>original/ (34 triangles)</i>											
Phong (No tree)	8059.0	4567.32	-	-	-	-	-	-	-	-	1x
Phong (Rec build)	28491.8	1299.03	2.0768	1711	18.2243	5.8768	92.7852	29.1717	0.0003888	0.9315	3.53x
Phong	15401.3	2391.43	0.7250	183	7.38043	6.9130	5.3438	30.8947	0.0002282	1.8284	1.91x
<i>long/ (30 triangles, textures)</i>											
Phong (No tree)	18362.6	3999.95	-	-	-	-	-	-	-	-	1x
Phong (Rec build)	58295.0	1259.98	0.0860	869	28.9287	3.6966	39.7344	14.6188	0.0003336	0.9140	3.17x
Phong	34366.6	2138.11	0.0878	73	5.7027	7.2973	2.19531	23.5477	0.0002032	0.9269	1.87x
<i>sphere/ (2188 triangles)</i>											
Phong (No tree)	1259320	55.0102	-	-	-	-	-	-	-	-	1x
Phong (Rec build)	178537.0	388.04	0.4661	625	28.8403	10.2460	44.6016	335.3833	0.0054411	0.5451	0.142x
GGX (Rec build)	178611.5	387.91	0.4359	625	28.8403	10.2460	44.6016	335.3750	0.0054054	0.5451	0.142x
Phong	27181.6	2555.70	39.6324	32467	19.2587	6.16219	898.066	18.4960	0.0008414	1.9375	0.022x
GGX	28631.7	2422.02	39.5324	32467	19.2587	6.16219	898.066	18.4959	0.0009029	1.9375	0.023x
Phong - NOTEX (mirror backwall)	36838.7	2544.96	42.7561	32467	19.2587	6.16219	898.066	16.3688	0.0009085	1.4390	0.029x
Phong - NOTEX (diffuse backwall)	24499.7	2827.71	45.2617	32467	19.2587	6.16219	898.066	18.4964	0.0007709	1.9375	0.019x
<i>bunny/ (69463 triangles)</i>											
Phong (Rec build)	8623940.0	7.74	30.4188	16694	34.5229	11.8253	1292.89	8357.9700	0.2192440	0.5693	1x
Phong	27121.2	2479.27	2756.5600	1885477	31.5396	4.5087	46064.2	16.9531	0.0004061	1.8893	0.0034x
GGX	25818.8	2586.38	1878.1200	1885477	31.5396	4.5087	46064.2	16.9516	0.0003987	1.8891	0.0032x
<i>dragon/ (871318 triangles)</i>											
Phong	74322.4	1159.48	25360.1000	19773123	33.2597	5.0403	503607	25.1965	0.0016568	3.1155	1x
GGX	75437.4	1144.00	26826.0000	19773123	33.2597	5.0403	503607	25.1945	0.0017115	3.1155	1.02x

- If the ray intersects the splitting plane after the exit point ($t_{max} \leq t_{split}$), it continues only to the closer node.
- If the ray intersects the splitting plane before the entry point ($t_{split} \leq t_{min}$), it continues directly to the further node.
- If the ray passes through both nodes, the further node is saved on the stack (together with the shortened limits t_{split} and t_{max}), and the ray continues to the closer node first.

When a leaf node is reached, it iterates over the array of stored indices and calculates intersections with all present triangles. Thanks to ordering the traversal from closest to furthest, the search can often be successfully terminated right in the first hit leaf.

4 Measurement results

The measurement was performed on the Original, long, Sphere (with texture), and Sphere (mirror backwall) scenes, and the resulting renders can be seen in 2, 3. I rendered 800x800px images, recursion depth 8, number of samples per light 32, and when GGX was turned on, ACESFilm with 1.25 exposure was used. In table 1, we can see the render measurement results; all times are in ms, and the Time vs Base result represents a time comparison against the scene without using an acceleration structure, if possible. Unless stated otherwise, the SAH kd-tree was used.

For simpler scenes (original and long), we can see that the total time was faster without a kd-tree, and with it, it was about 2-3 times slower.

On the sphere scene, we can already see that the kd-tree helps significantly, and the median build was 7x faster on the scene with textures. When I used the SAH build, it was 45x faster. If I put a non-reflective/diffuse wall instead of a texture, the speedup was 52x, and the reflective/mirror wall was slower than the texture with a normal map, being 34x faster.

For the bunny scene, it was no longer computable without a kd-tree, but even the median build was barely enough, and compared to it, the SAH build achieved a 312.5x speedup.

The dragon scene was beyond my computing power without a tree or even with the median build. At least it shows that the slowdown when using GGX instead of Phong is 2%.

5 Solved problems

The first problem I encountered appeared when switching from a large scene with boxes to a small scene with spheres. There I had to start solving problems with different epsilons to render at least something. One epsilon for comparing floats was not enough. A second epsilon for shifting the emitted rays away from the triangle helped, but what really helped was an epsilon that stopped the shadow ray early enough before the light.

There was also quite a problem with normal maps and mipmapping.

With mipmaps, I dealt with the fact that I didn't have a scene to test it on, and the one I created hid a lot of problems and hours of reworking. Besides that, finding a method to determine the mipmap level of a given texture in the scene was a tough nut to crack, see Mipmapping.

With normal maps, I had to understand where tangents come from, how to work with them later, and how to keep them always orthonormal to the normal.

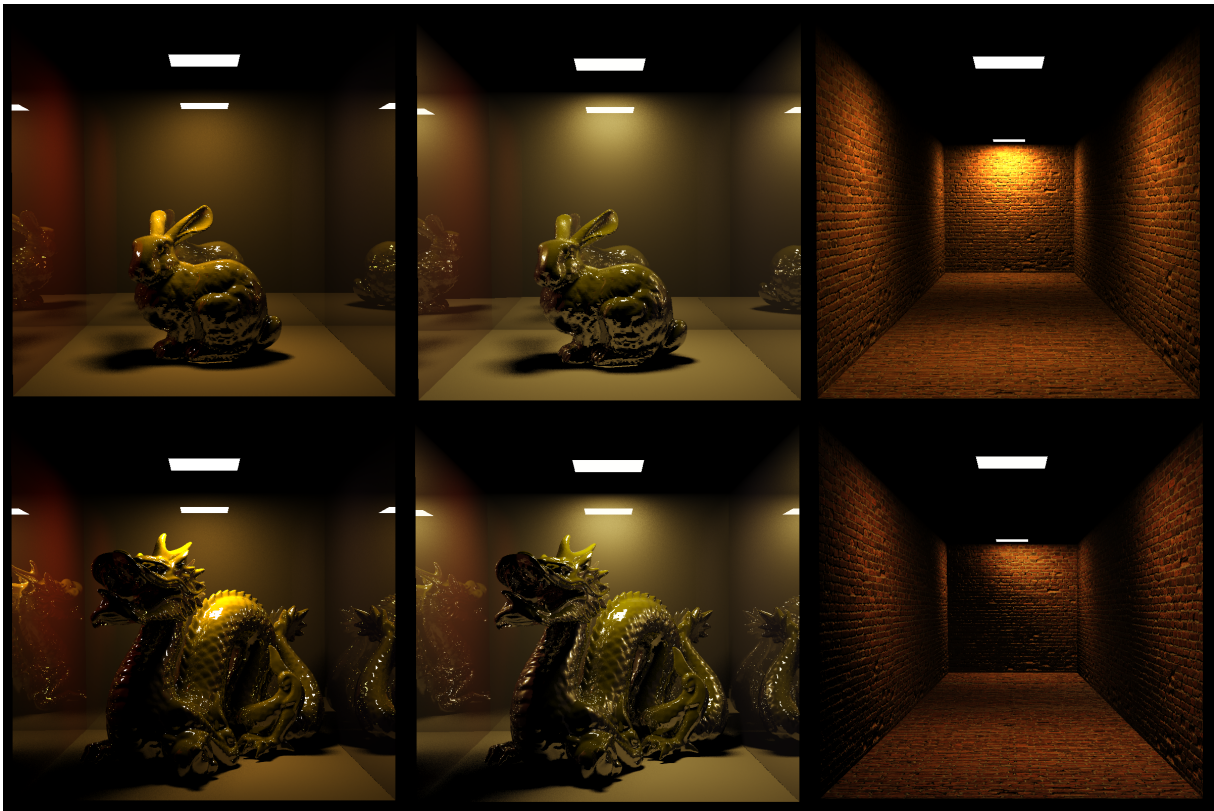


Figure 2: Scenes: Bunny, Dragon, Long

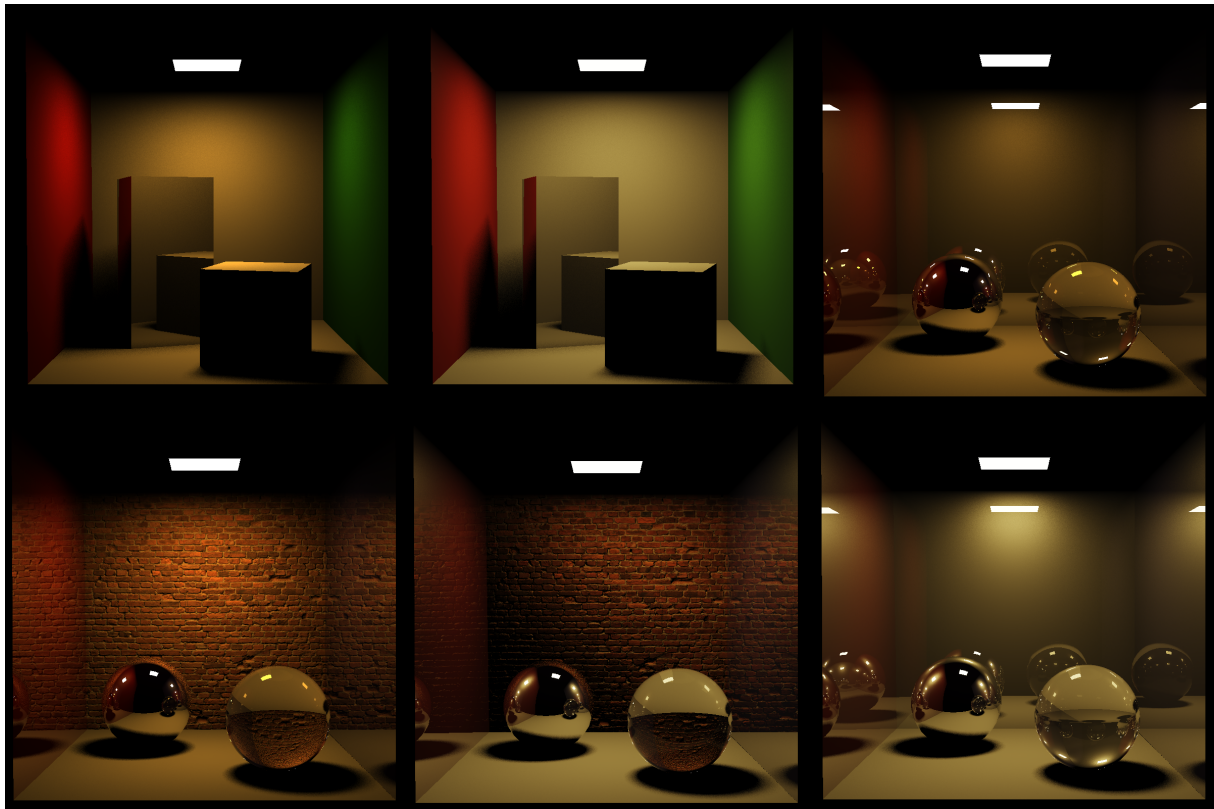


Figure 3: Scenes: Original(Box), Spheres(Textured), Spheres(Mirror backwall)